Universal Bank Personal Loan Classification

OM 484 Final Project Report

Kendra Kershaw Jacob Hutchinson Eliott Humes Stephanie Linares

April 2026

Contents

Executive Summary

Universal Bank's last personal loan campaign reached every customer in the sample and converted only 9.6 percent of them. Most contacts produced nothing, which is wasteful when contact cost is the same regardless of the outcome. Our project asks a simpler question: can the bank predict, in advance, which customers are likely to accept a personal loan offer?

We worked with a 5,000 customer dataset that includes age, income, family size, education, credit card spending, and several account flags. After cleaning the file and dropping two columns that carried no information (customer ID and ZIP code), we split the data 70 / 30 into training and test sets. Forward stepwise selection on a logistic regression chose nine of the eleven candidate features. We then trained three models on those features: logistic regression, a pruned classification tree, and a single hidden layer neural network.

On the 1,500 test customers the decision tree performed best, with 98.5 percent accuracy, 92.9 percent precision on the positive class, and 91.0 percent recall. The neural network came second with the highest recall of the three (93.8 percent) but more false positives. Logistic regression trailed both, missing roughly a quarter of real acceptors, which is the behavior we expected from a linear model on a dataset where the boundary between acceptors and decliners is shaped by interactions among income, education, and family size.

The practical reading is that a campaign guided by the decision tree would contact roughly 9 percent of the customer base and still capture more than nine out of every ten genuine acceptors. The tree also offers something the neural network cannot: rules a marketing team can read and audit. The two strongest segments are higher income customers with a graduate degree, and higher income undergraduates with larger families.

We recommend deploying the rule-based ranking model as the primary scoring engine for personal loan campaigns. Its segmentation rules read directly so marketing can audit and explain who is being contacted and why. Under representative cost assumptions, the model converts a customer for roughly a tenth of the per-acceptor marketing spend of a blanket campaign, freeing both budget and contact capacity for other use. The contact threshold should be tuned to the bank's actual per-contact cost, and the model should be retrained after each campaign cycle as customer behaviour shifts.

1. Problem Statement

Universal Bank earns most of its revenue from depositors but a personal loan customer is more profitable per relationship. The bank has run blanket loan campaigns that contact every customer in the book. The conversion rate on the most recent campaign was 9.6 percent. That number is reasonable for an untargeted offer but it implies that nine out of every ten contacts did not convert. Contact cost is fixed, so the dominant effect of a blanket campaign is wasted spend.

The analytical task is binary classification. Given a customer's banking profile and demographics, the model should output whether that customer is likely to accept a personal loan. A useful model would let the bank focus outreach on a smaller, higher quality set of prospects, which raises the hit rate per contact without raising marketing budget.

The class imbalance in the data shapes how we evaluated the models. Figure 1 shows that 480 of 5,000 customers (9.6 percent) accepted the previous offer. A model that always predicts "no" already scores 90.4 percent accuracy on this dataset, so accuracy by itself is not a useful target. We focus on precision and recall on the positive (accepted) class, and on F1 as a single combined number.

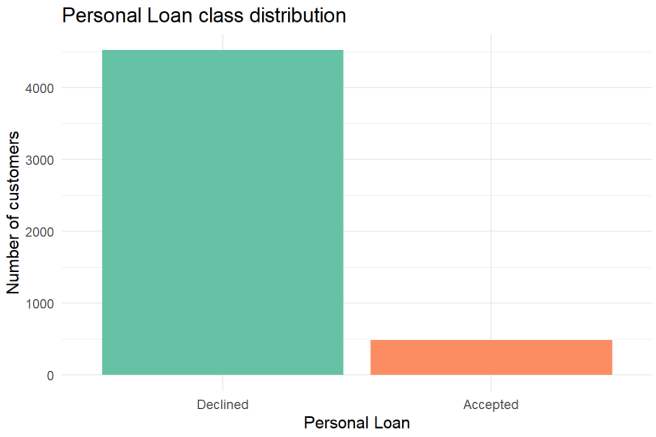


Figure 1. *Class distribution. The positive class is rare, which means accuracy alone is misleading.*

2. Data Source

The dataset is the Universal Bank retail banking sample provided in `UniversalBank.csv`. It contains 5,000 customer records and 14 columns, including the binary target `Personal Loan` (1 if the customer accepted the offer, 0 otherwise). Table 1 lists each variable and how we treat it.

The file has no missing values in any column. The mix of continuous, ordinal, and binary variables fits both linear and tree based classifiers without much extra work.

3. Data Investigation

We started by looking at the distributions of the four continuous features that drive most of the modelling decisions (Figure 2). Income and CCAvg are right skewed with long upper tails. Age is close to uniform between 23 and 67. Mortgage is strongly zero inflated because more than half of customers report no mortgage. The skew in Income and CCAvg is not a data quality issue; it reflects the actual wealth distribution of the customer base. Tree based models handle skewed inputs natively, so we did not transform these columns.

Table 2 reports summary statistics after cleaning. Two patterns are worth flagging. First,

Table 1. *Dataset variables.*

Variable	Type	Description
ID	Identifier	Row counter, dropped before modelling
Age	Continuous	Customer age in years
Experience	Continuous	Years of professional work experience
Income	Continuous	Annual income in thousands of dollars
ZIP Code	Categorical	Five digit postal code, dropped
Family	Ordinal	Family size from 1 to 4
CCAvg	Continuous	Monthly credit card spend (in thousands)
Education	Ordinal	1 = Undergrad, 2 = Graduate, 3 = Advanced
Mortgage	Continuous	Mortgage value in thousands
Securities Account	Binary	Has a securities account
CD Account	Binary	Has a certificate of deposit account
Online	Binary	Uses online banking
CreditCard	Binary	Has a Universal Bank credit card
Personal Loan	Binary	Target: customer accepted the offer

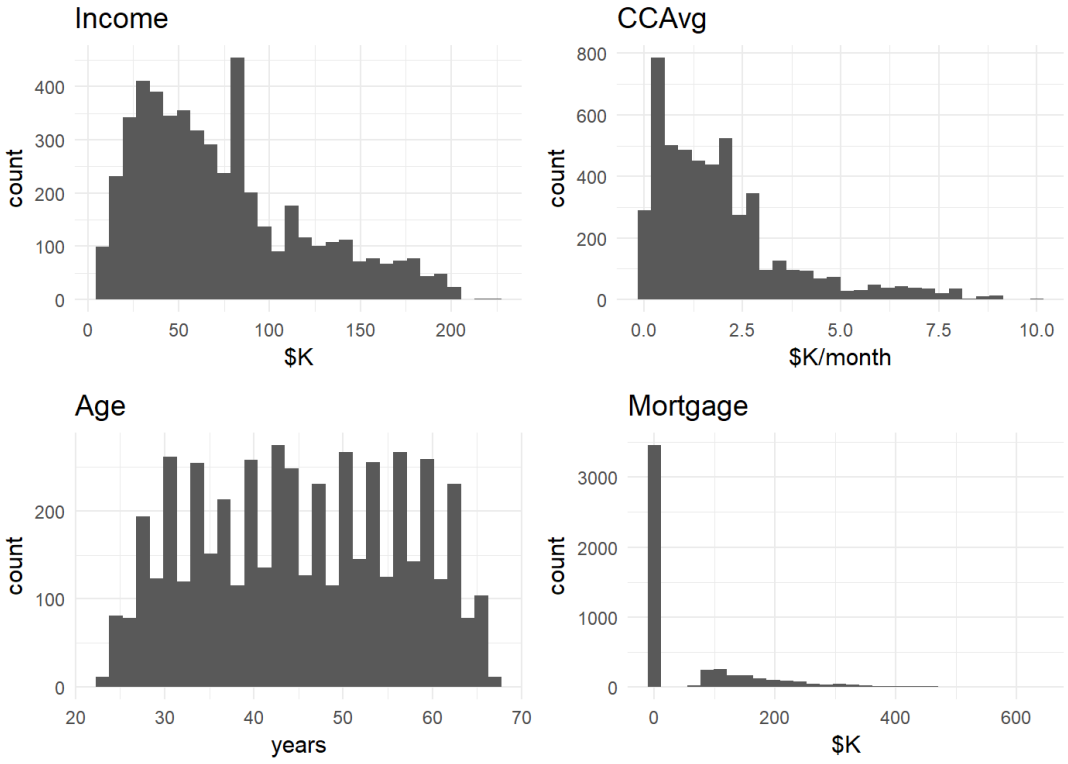


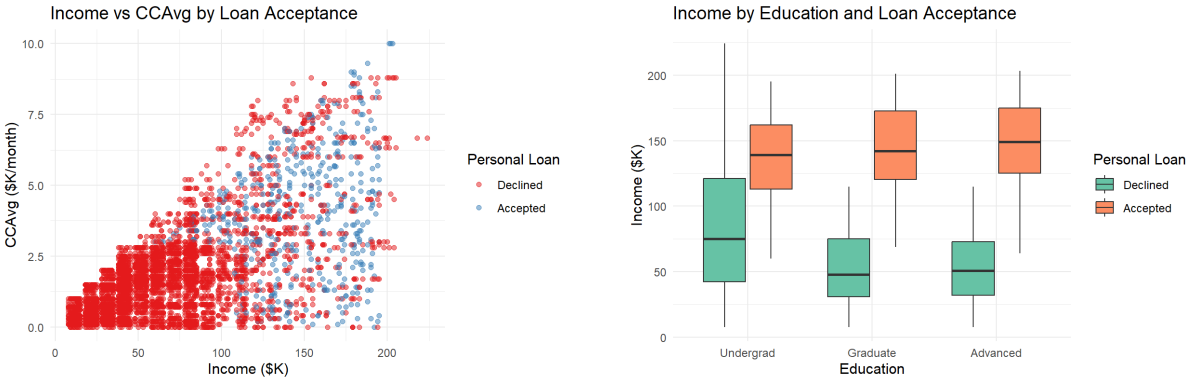
Figure 2. *Distributions of Income, CCAvg, Age, and Mortgage.*

Age and Experience track each other almost exactly (Experience is roughly Age minus 25), which means only one of them carries unique information. Second, the spread on Mortgage is huge relative to its median because the median is zero.

Table 2. *Summary statistics for the continuous variables.*

Feature	Mean	Std	Min	Q1	Median	Max
Age	45.3	11.5	23	35	45	67
Experience	20.1	11.4	0	10	20	43
Income	73.8	46.0	8	39	64	224
CCAvg	1.9	1.7	0	0.7	1.5	10
Mortgage	56.5	101.7	0	0	0	635

To see how the strongest predictors separate the two classes, we plotted Income against monthly credit card spend (Figure 3a) and Income against Education level split by acceptance (Figure 3b). In both views the acceptors cluster heavily at the high income end. Education matters as well: at any fixed income level, a customer with a graduate or advanced degree is more likely to accept than a customer with an undergraduate degree.



(a) *Income vs. monthly credit card spend, coloured by loan acceptance.*

(b) *Income by education level, split by loan acceptance.*

Figure 3. *Income separates acceptors from decliners at high values, with Education adding a second axis of separation.*

4. Data Cleansing

The cleaning steps in `bank_loan.R` (lines 41 to 58) are:

1. Drop `ID`. It is just a row counter and carries no predictive information.

2. Drop `ZIP Code`. There are too many distinct values (around 467) to use it as a categorical without first grouping codes into geographic clusters, which is out of scope here.

3. Replace spaces in column names with underscores so R formulas parse cleanly.

4. Clip negative `Experience` values (52 rows) to zero. Negative years of experience is impossible; this is a data entry artifact.

5. Confirm there are no missing values across the remaining columns (`sum(is.na(bank))` returns 0), so no imputation is needed.

6. Convert `Personal_Loan`, `Education`, `Family`, and the four binary account flags to factors. Treating `Education` as a factor matters: as an integer R would assume the gap from Un-

dergrad to Graduate is the same "size" as the gap from Graduate to Advanced, which the logistic model would interpret literally.

For the neural network we also standardize the design matrix to zero mean and unit variance, fitting the scaler on the training set and reusing the same numbers on the test set. Fitting the scaler on the full dataset would leak test set statistics into training. The decision tree uses the raw values because it is invariant to monotonic rescaling.

To address the 9.6 percent class imbalance for the neural network, we used random oversampling on the training set only with the `ROSE::ovun.sample` function. The test set was left at its natural class proportions so the test metrics reflect real population performance. Logistic regression and the decision tree were trained on the unbalanced data because both methods handle imbalance reasonably well at default settings.

5. Feature Selection

We used forward stepwise selection on a logistic regression to choose features (`bank_loan.R`, lines 96 to 108). The procedure starts from a model that contains only an intercept and adds, at each step, the feature that most reduces the Akaike Information Criterion (AIC). It stops when no remaining feature lowers AIC any further. The benefit of this approach is that it picks features based on how much they actually contribute to model fit, while penalizing complexity through AIC.

The procedure selected nine of the eleven candidate features:

- Income
- Education
- Family
- CD_Account
- CreditCard
- CCAvg
- Online
- Securities_Account
- Mortgage

The two features that were not selected are Age and Experience. Dropping both makes sense given the near perfect correlation between them that we noted earlier; once Income was already in the model neither contributed enough additional signal to lower AIC further. Mortgage was retained, but its coefficient is small ($p = 0.11$); it sits at the borderline of the selection criterion and contributes less than the other eight features.

We used the same nine feature set across all three models so that performance differences reflect the model itself, not differences in input information.

6. Model Development and Evaluation

6.1 Setup

The cleaned 5,000 row dataset was split 70 / 30 into training (3,500 rows) and test (1,500 rows) using `caret::createDataPartition`, which performs a stratified split on `Personal_Loan` so the 9.6 percent acceptance rate is preserved exactly in both folds. With this much class imbalance a plain random split could shift the test acceptor count by several points run-to-run, which would propagate directly into the recall metric; stratification removes that source of variance. `set.seed(123)` is used for reproducibility, and the test set is used only for final scoring and is not seen by any model during fitting or feature selection.

We trained three models on the nine selected features:

- **Logistic regression** (`glm`, binomial family, inherited from the stepwise procedure).
- **Decision tree** (`rpart`, grown deep with `minsplit = 20` and `cp = 0.001`, then pruned at the complexity parameter that minimizes the cross validated error column `xerror`).
- **Neural network** (`nnet` with one hidden layer of 5 nodes, weight decay 0.001, up to 1,000 epochs), trained on the oversampled and standardized training set.

For each model we report accuracy, precision (positive class), recall (positive class), and F1 from `caret::confusionMatrix` at a classification threshold of 0.5.

6.2 Test Set Performance

Table 3 summarizes the four metrics for each model on the 1,500 row test set, and Figure 4 shows the underlying confusion matrices.

Table 3. *Test set performance (n = 1,500, 144 acceptors).*

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9640	0.8462	0.7639	0.8029
Decision Tree	0.9847	0.9291	0.9097	0.9193
Neural Network	0.9727	0.8084	0.9375	0.8682

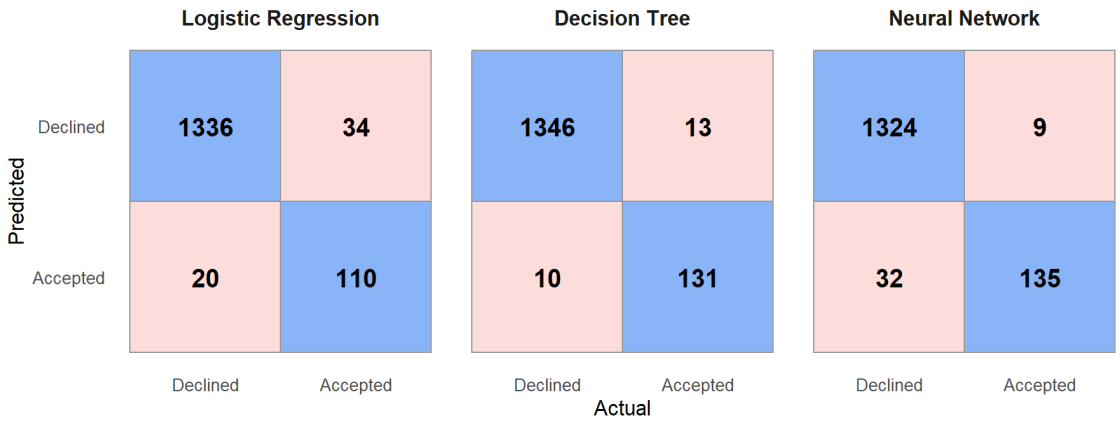


Figure 4. *Test-set confusion matrices. Rows: predicted class. Columns: actual class. Blue cells are correct predictions; pink cells are errors.*

The decision tree comes out ahead on accuracy, precision, and F1. The neural network has the highest recall (0.938, catching 135 of 144 acceptors) but produces noticeably more

false positives (32 versus 10), which drags its precision down to 0.81. Logistic regression sits between the two on precision but has the weakest recall (0.764, missing roughly a quarter of the actual acceptors). With this much class imbalance the recall figure is the most consequential, because every missed acceptor is a customer the bank should have contacted; the decision tree's combination of high recall and high precision is what makes it the front-runner.

6.3 Why Logistic Regression Underperforms

The data investigation suggested that acceptance depends on interactions between income, education, and family size, not on any single linear combination of those features. A logistic regression assumes the log odds of acceptance is a weighted sum of the inputs, which cannot represent "a high income customer is much more likely to accept if they also have a graduate degree" as a single coefficient. The decision tree captures that kind of multiplicative structure naturally because each split conditions on the splits above it. The neural network can also represent interactions, but with only one hidden layer of five nodes and an oversampled training set, it ends up trading some precision for recall relative to the tree.

6.4 Decision Tree Structure

Because the pruned tree is small enough to read directly, we include it as Figure 5. Each leaf shows the proportion of training customers that landed there and the predicted class. Two leaves account for most of the predicted acceptors:

- Customers with Income at or above roughly \$115K and Education at Graduate or Advanced level: high acceptance probability.
- Customers with Income at or above roughly \$115K, an Undergraduate degree, and a Family size of three or more: high acceptance probability.

A smaller third leaf flags mid income customers (Income roughly \$93–107K) with a graduate or advanced degree and elevated monthly credit card spend (CCAvg at or above roughly \$3K/month).

6.5 Neural Network Structure

The neural network has the architecture shown in Figure 6. The input layer holds one node per dummy column produced by `model.matrix` on the nine selected features (so each factor level becomes its own input). After dummy encoding of Education (three levels) and Family (four levels), this expands to 13 input nodes. The hidden layer has five sigmoid nodes, and the output is a single node giving the predicted probability of acceptance, for a 13-5-1 architecture with 76 weights including biases.

7. Practical Implications

The model output is a probability between 0 and 1, not a hard label. The bank gets to choose where to set the threshold, and that choice is a business decision rather than a statistical one. With the decision tree at threshold 0.5, the bank would contact roughly 9 percent of the customer base (141 customers out of 1,500 in our test slice) and capture 131 of the 144 genuine acceptors. That is a 91 percent capture rate on fewer than a tenth of the contacts, compared with a blanket campaign that contacts 100 percent of the base to reach the same group.

For the marketing team the tree rules are easier to act on than either the logistic regression coefficients or the neural network weights. The two primary segments are:

1. Customers with Income at or above \$115K and a graduate or advanced degree.

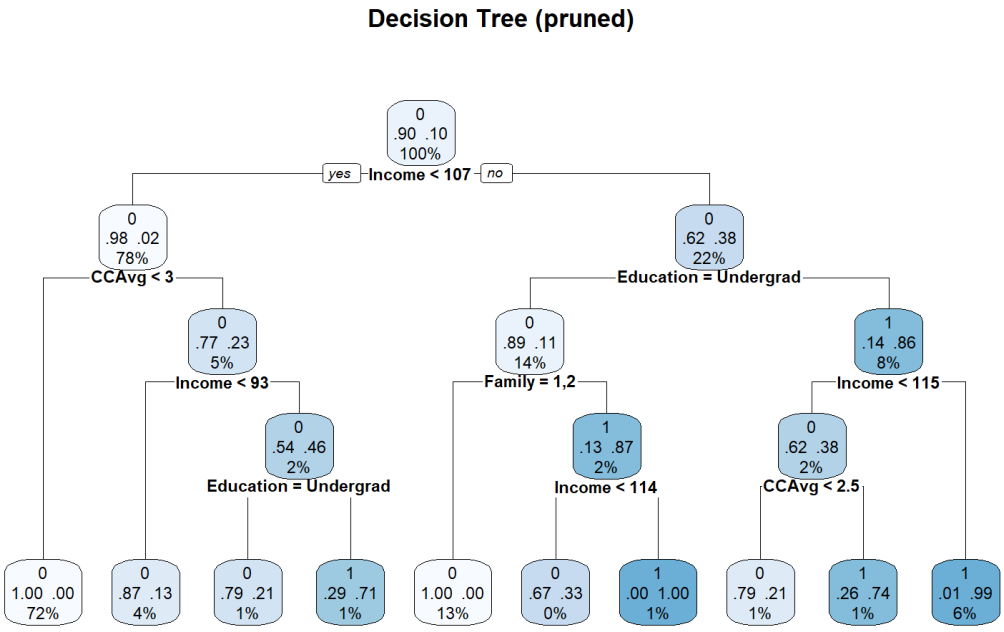


Figure 5. *Pruned decision tree. Each leaf shows the predicted class, the predicted probability, and the percentage of training customers that fall into the leaf.*

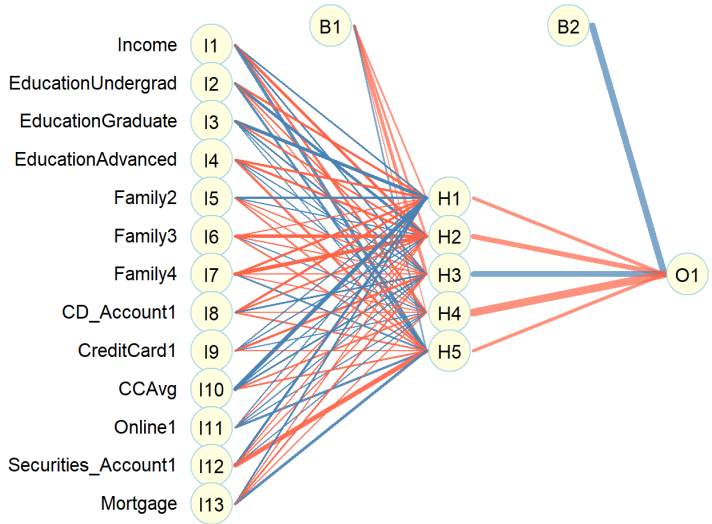


Figure 6. *Neural network architecture. Blue lines are positive weights, red lines are negative; line thickness is proportional to weight magnitude.*

2. Customers with Income at or above $115K, an undergraduate degree, and a family of three or more.

A third segment captures mid income customers with a graduate or advanced degree and above average credit card spend. All nine inputs the model uses are fields the bank already collects, so deployment does not require any new data pipelines. Scoring can be computed in batch before each campaign or applied on the fly when a customer interacts with a channel.

7.1 Cost-Benefit at Scale

To put the model in dollar terms, we ran an illustrative cost-benefit exercise on the full 5,000 customer base. We assume a per-contact cost of $10 (a representative cost for an outbound call campaign) and a net profit of $2,000 per converted personal loan over the loan's lifetime. Both numbers are illustrative and would be replaced with the bank's actual figures in deployment, but they are the right order of magnitude.

Table 4. *Illustrative cost-benefit comparison at the 0.5 threshold.*

	Blanket campaign	Targeted (DT)
Contacts	5,000	∼470
Contact cost	$50,000	$4,700
Acceptors reached	480	∼437
Revenue (at $2,000 each)	$960,000	$874,000
Net profit	$910,000	$869,300
Cost per acceptor	$104	$10.75

The two campaigns end up within 4.5 percent of each other on net profit, because the targeted campaign misses roughly 43 acceptors at this threshold. The real win is on the *cost per acceptor* line: the targeted campaign converts a customer for one tenth of the marketing spend. That is what frees capacity. The 4,530 contacts the bank does not make can be redirected to other product offers, used to follow up with high probability prospects more aggressively, or simply not made, which reduces customer offer fatigue.

If contact cost rises (say, a $25 in-branch interaction instead of a $10 call) or the loan margin shrinks, the cost ratio shifts. The 0.5 threshold is a default rather than a constraint: lowering it raises recall at the expense of some false alarms, and raising it does the opposite, so marketing can tune it to whichever cost structure applies without retraining the model.

8. Conclusion and Recommendations

We trained three classifiers on a 5,000 customer sample and evaluated them on a held out test set of 1,500 customers. The pruned decision tree was the strongest model overall, with 98.5 percent accuracy, 92.9 percent precision on the positive class, and 91.0 percent recall. The neural network had higher recall (93.8 percent) but lower precision, and logistic regression underperformed because the boundary between acceptors and decliners is not linear in the input features.

Concrete recommendations for Universal Bank:

1. Use the decision tree as the primary scoring model for personal loan campaigns. It performs best on the metrics that matter most under class imbalance, and the rules are simple enough for the marketing team to read and audit.

2. Keep the neural network as a secondary reference. It catches slightly more acceptors when recall matters more than precision, and it is a useful sanity check on the tree's output.

3. Target the two primary customer segments first (high income customers with graduate or advanced degrees, and high income undergraduates with larger families) on the next campaign.

4. Tune the classification threshold to the bank's actual cost structure before going live, instead of using the 0.5 default.

5. Retrain the model after each campaign cycle, or sooner if there is a notable shift in customer mix or macroeconomic conditions.

One honest caveat is worth flagging: we did not engineer additional features (income to credit card spend ratios, education and family interactions, and so on), which a production version of this model would likely benefit from.

A. Complete R Code

The full `bank_loan.R` script is included below for reference. It covers data loading, cleaning, exploratory plots, the 70 / 30 split, forward stepwise selection, all three model fits, and the final performance comparison.

```r
# bank_loan.R - Universal Bank personal loan classification
# Models: logistic regression, decision tree, ANN
# Methodology follows Lab 6 (LR + stepwise), Lab 7 (CART), Lab 8 (ANN)

# install.packages("caret")           # uncomment if not installed
# install.packages("rpart")
# install.packages("rpart.plot")
# install.packages("nnet")
# install.packages("NeuralNetTools")
# install.packages("ROSE")
# install.packages("ggplot2")
# install.packages("gridExtra")

library(caret)
library(rpart)
library(rpart.plot)
library(nnet)
library(NeuralNetTools)
library(ROSE)
library(ggplot2)
library(gridExtra)

set.seed(123)


# load
bank <- read.csv("UniversalBank.csv", header = TRUE, check.names = FALSE)
str(bank)
head(bank)
summary(bank)

# distributions of the four continuous features that drive the modelling
h1 <- ggplot(bank, aes(Income))    + geom_histogram(bins = 30) + labs(title = "
    Income",    x = "$K")       + theme_minimal()
h2 <- ggplot(bank, aes(CCAvg))     + geom_histogram(bins = 30) + labs(title = "
    CCAvg",     x = "$K/month") + theme_minimal()
```

```r
h3 <- ggplot(bank, aes(Age))      + geom_histogram(bins = 30) + labs(title = "Age ",      x = "years")    + theme_minimal()
h4 <- ggplot(bank, aes(Mortgage)) + geom_histogram(bins = 30) + labs(title = "Mortgage", x = "$K")       + theme_minimal()
grid.arrange(h1, h2, h3, h4, ncol = 2)


# cleaning
bank$ID          <- NULL                      # row counter
bank$`ZIP Code` <- NULL                       # too many distinct values
names(bank) <- gsub(" ", "_", names(bank))
bank$Experience[bank$Experience < 0] <- 0  # 52 rows are negative, clip to 0
sum(is.na(bank))                              # 0 - no NAs

bank$Personal_Loan <- factor(bank$Personal_Loan, levels = c(0, 1))
table(bank$Personal_Loan)
prop.table(table(bank$Personal_Loan))       # ~9.6% acceptance, very imbalanced

# treat the obvious categoricals as factors so LR doesn't assume linear spacing
bank$Education          <- factor(bank$Education, levels = c(1, 2, 3),
                                  labels = c("Undergrad","Graduate","Advanced"))
bank$Family             <- factor(bank$Family, levels = c(1, 2, 3, 4))
bank$Securities_Account <- factor(bank$Securities_Account, levels = c(0, 1))
bank$CD_Account         <- factor(bank$CD_Account, levels = c(0, 1))
bank$Online             <- factor(bank$Online, levels = c(0, 1))
bank$CreditCard         <- factor(bank$CreditCard, levels = c(0, 1))

# class distribution
ggplot(bank, aes(Personal_Loan, fill = Personal_Loan)) +
  geom_bar() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Personal Loan class distribution",
       x = "Personal Loan", y = "Number of customers") +
  theme_minimal() + theme(legend.position = "none")

# Income vs CCAvg colored by acceptance
ggplot(bank, aes(Income, CCAvg, color = Personal_Loan)) +
  geom_point(alpha = 0.5) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Income vs CCAvg by Loan Acceptance",
       x = "Income ($K)", y = "CCAvg ($K/month)",
       color = "Personal Loan") +
  theme_minimal()

# Income by Education, split by acceptance
ggplot(bank, aes(Education, Income, fill = Personal_Loan)) +
  geom_boxplot() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Income by Education and Loan Acceptance",
       x = "Education", y = "Income ($K)", fill = "Personal Loan") +
  theme_minimal()


# train / test split (70/30, stratified on Personal_Loan)
# stratified split preserves the 9.6% acceptance rate in both folds so the
# test acceptor count does not swing run-to-run with this much imbalance.
idx   <- createDataPartition(bank$Personal_Loan, p = 0.7, list = FALSE)
train <- bank[idx,  ]
test  <- bank[-idx, ]
table(train$Personal_Loan)
table(test$Personal_Loan)


```

```r
# forward stepwise feature selection (Lab 6)
null_mod <- glm(Personal_Loan ~ 1, data = train, family = binomial)
full_mod <- glm(Personal_Loan ~ ., data = train, family = binomial)

step_forw <- step(null_mod,
                  scope = list(lower = null_mod, upper = full_mod),
                  direction = "forward",
                  trace = 0)
step_forw
summary(step_forw)

selected <- attr(terms(step_forw), "term.labels")
selected
sel_formula <- as.formula(paste("Personal_Loan ~", paste(selected, collapse = " + ")))


# logistic regression (Lab 6) - step_forw is already a glm, use it directly
lr_mod <- step_forw

prob_lr <- predict(lr_mod, newdata = test, type = "response")
pred_lr <- factor(ifelse(prob_lr >= 0.5, "1", "0"),
                  levels = levels(test$Personal_Loan))
cm_lr <- confusionMatrix(pred_lr, test$Personal_Loan, positive = "1")
cm_lr


# decision tree (Lab 7) - grow deep then prune at min xerror cp
dt_mod <- rpart(sel_formula, data = train, method = "class",
                control = rpart.control(minsplit = 20, cp = 0.001))
best_cp   <- dt_mod$cptable[which.min(dt_mod$cptable[,"xerror"]), "CP"]
dt_pruned <- prune(dt_mod, cp = best_cp)

rpart.plot(dt_pruned, type = 2, extra = 104, fallen.leaves = TRUE,
           cex = 0.7, box.palette = "Blues",
           main = "Decision Tree (pruned)")

pred_dt <- predict(dt_pruned, newdata = test, type = "class")
pred_dt <- factor(pred_dt, levels = levels(test$Personal_Loan))
cm_dt   <- confusionMatrix(pred_dt, test$Personal_Loan, positive = "1")
cm_dt


# ANN (Lab 8)
# 1) oversample TRAIN only (anti-leakage)
# 2) scale predictors with TRAIN means/SDs, reapply same numbers to TEST
# 3) fit nnet, predict, threshold at 0.5

train_sel <- train[, c(selected, "Personal_Loan")]
test_sel  <- test[ , c(selected, "Personal_Loan")]

train_ann <- ovun.sample(Personal_Loan ~ ., data = train_sel,
                         method = "over", seed = 123)$data
table(train_ann$Personal_Loan)

# factors -> 0/1 dummy columns so nnet has a numeric matrix to chew on
x_train_raw <- model.matrix(Personal_Loan ~ . - 1, data = train_ann)
x_test_raw  <- model.matrix(Personal_Loan ~ . - 1, data = test_sel)

mu       <- colMeans(x_train_raw)
sd_train <- apply(x_train_raw, 2, sd)
x_train  <- scale(x_train_raw, center = mu, scale = sd_train)
x_test   <- scale(x_test_raw,  center = mu, scale = sd_train)
```

```r

y_train <- as.numeric(train_ann$Personal_Loan) - 1   # factor "0"/"1" -> 0/1

ann_mod <- nnet(x = x_train, y = y_train,
                size = 5, decay = 0.001, maxit = 1000,
                linout = FALSE, trace = FALSE)
ann_mod$n
length(ann_mod$wts)
summary(ann_mod)

# visualise the trained network
par(mar = c(2, 8, 2, 2))
plotnet(ann_mod, alpha = 0.7,
        circle_col = "lightyellow",
        pos_col = "steelblue", neg_col = "tomato",
        bias = TRUE, node_labs = TRUE, var_labs = TRUE,
        pad_x = 0.7)

prob_ann <- predict(ann_mod, newdata = x_test)
pred_ann <- factor(ifelse(prob_ann >= 0.5, "1", "0"),
                   levels = levels(test$Personal_Loan))
cm_ann <- confusionMatrix(pred_ann, test$Personal_Loan, positive = "1")
cm_ann


# compare the three models
results <- rbind(
  LR  = c(cm_lr$overall["Accuracy"],  cm_lr$byClass[c("Precision","Recall","F1")
      ]),
  DT  = c(cm_dt$overall["Accuracy"],  cm_dt$byClass[c("Precision","Recall","F1")
      ]),
  ANN = c(cm_ann$overall["Accuracy"], cm_ann$byClass[c("Precision","Recall","F1")
      ])
)
print(round(results, 4))
```